Loop Unrolling in High-Level Synthesis: Controller Delay, Resource Cost, and an FPGA Experiment

Christian Percival
Electronic Engineering
Hochschule Hamm-Lippstadt
percival.christian@gmail.com

***Abstract*—Loop unrolling uncovers parallelism as a result of multiple loop iterations merged into one massive loop body. In software, this technique can help lessen the burden of loop control, but in case of high-level synthesis (HLS), it impacts on physical data path, resource sharing, diversity of multiplexers as well as finite-state machine (FSM) controller. The main paper authored by Kurra, Singh and Panda thus considers unroll-factor selection a timing-aware design problem, because it is possible to reduce the amounts of cycles at the expense of proper controller delays. The paper gives an overview of the said paper and describes Vitis HLS experiment based on 32-element integer dot product. The unroll factors from 1 to 32 were synthesized two times: at first, using ordinary memories, and the second time, employing complete partitioning. In the first case, unroll factor 2 has helped reduce the top-level latency from 370 ns to 210 ns; however, it should be noted that unroll factors from 4 to 32 had no positive effect since the initiation interval increased and, as a result, the case of full unrolling faced a challenge with timing. In the second case, i.e. when complete partitioning was utilized, latency constituted around 50 ns at unroll factor 320 with estimated resource usage equal to 60 DSPs, 4234 flip-flops and 2150 LUTs. The obtained results do not indicate FSM delay in a direct way, but they provide proof for the major engineering compromise represented in the paper mentioned above: aggressive unrolling is only beneficial when the controller timing allows to make full use out of the parallelism revealed.**

***Index Terms*—high-level synthesis, loop unrolling, controller delay, finite-state machine, FPGA, initiation interval, array partitioning**

I. Introduction

The process known as high-level synthesis is responsible for the transformation of a behavioral model described in either the C language or the C++ programming language into an implementation based on the register-transfer level (RTL) which contains transfer registers, specific combinational logic implementations, integrated memory units, as well as multiplexers and corresponding controller logic [1].

An example of such a transformation can be represented by the method called loop unrolling which is responsible for the duplication of certain blocks of code within a loop that allows discovering new operations performed during different iterations of a process, which results in making the number of controllable decisions lower and increases the number of parallel processes. However, the authors Kurra et al. pointed out that enlarging the data flow graph would lead to increasing the complexity of the constructed finite-state machine (FSM) controller [2]. Thus, the significant question is whether the application of loop unrolling leads to an improvement in terms of latency or the selection of the unrolling factor provides a reduction of latency without going beyond the controller-time budget.

This seminar paper follows the required academic structure consisting of mathematical foundations of the selected problem, process of the transformation itself, the discussion of algorithmic complications, and application of the C/C++ language. At first, the definition of the term "loop unrolling" and the definition of "controller timing" is given; then the general description of the delay estimation process, according to the claims presented in the core paper, is offered, and, finally, the comparison of the identified theory with the results of a modern FPGA experiment is provided.

II. Formal Background

A. Loop-Unrolling Transformation

Consider a counted loop with trip count N and body $B(i)$:

$$\texttt{for } i = 0, \ldots, N-1: \quad B(i). \tag{1}$$

For an unroll factor u, the transformed loop executes up to u original iterations in each new iteration:

$$i = 0, u, 2u, \ldots, \qquad B(i), B(i+1), \ldots, B(i+u-1). \tag{2}$$

Ignoring a possible remainder loop, the new trip count is

$$N_u = \left\lceil \frac{N}{u} \right\rceil. \tag{3}$$

The algorithmic work that is performed is equivalent to N. Therefore, a linear loop would still be $\mathcal{O}(N)$ regardless of the unfolding because it merely causes the addition of a constant value and more parallelism than a particular complexity class. If there were m operation nodes in the original body of code, the unfolded body of code features approximately um nodes. For this reason, the operations of HLS scheduler and binder will operate a much larger local graph even though the original number of operations has not changed.

The transformation can be performed by hand. HLS pragma needs to be performed at the time of synthesizing:

```
loop_main:
for (int i = 0; i < 32; ++i) {
#pragma HLS UNROLL factor=4
   q += x[i] * z[i];
}
```

The pragma does not get executed at run time but rather informs HLS compiler that it is necessary to unfold and schedule four iterations at once. Whether these four operations get executed simultaneously or not will depend on the dependencies, availability of multipliers, memory ports, binding decisions, and the limit that is defined by the clock constraint [1], [3]

B. Latency, Clock Period, and Throughput

A shorter schedule in clock cycles is not sufficient to guarantee a faster circuit. The execution time is

$$T_{\text{exec}}(u) = L_{\text{cycles}}(u)\, T_{\text{clk}}(u), \tag{4}$$

where L_{cycles} is cycle latency and T_{clk} is the implemented clock period. The initiation interval is defined as the number of cycles that take place between starting iterations of the loop and equals to 1 for pipelined loops – in this case a new iteration would take place every cycle. The initiation interval of 4 would require an iteration to be executed after 4 cycles. So reducing the number of trips can be compensated with the increasing initiation interval.

The High-Level Synthesis (HLS) timing path addressed in the article by Kurra et al. consists of several modules, like the controller's next-state/output logic, input multiplexers and more [2]. A schedule can be considered correct only when the total delay does not exceed the chosen clock period.

III. Controller Delay and Unroll-Factor Selection

A. Cause of Controller Delay Growth

The unrolling process will grow the data-flow graph and require more nodes to be mapped to both the control steps and the functional units. With resource sharing, the number of operations that may use the same functional unit can also grow and, thus need more multiplexers to select between operations. The study also states that the number of states in the FSM can also increase from four states of the schedule to six after one unroll. More states require a larger state register and more complex combinational decoding and next-state logic. [2].

The controller-delay estimate used by the paper is

$$D_{\text{FSM}} = A\, \log(N_{\text{op}})\, N_{\text{statebits}}, \tag{5}$$

with

$$A = x + yN_{\text{FUctrl}} + zN_{\text{var}}. \tag{6}$$

Notations in this equation are as follows N_{op} means the number of planned operations, $N_{\text{statebits}}$ stands for the number of bits in the state register, N_{FUctrl} is a number of the control signals in the functional unit, and N_{var} is the number of variables in the studied model. Constants x, y and z depend on the particular technology used. Equation 5 is an early estimate, not a replacement for final logic synthesis.

B. Search Overhead and the Main Contribution

For one iteration, it is possible to just test a small number of the factors. However, for many iterations, it is not possible to exhaustively examine all different combinations of the factors: if loop i has c_i factors, then the number of combinations of the factors is given by the $\prod_i c_i$ across all loops where i varies from 1 to n. Each combination will incur the processes of scheduling, RTL implementation, and logic synthesis.

The three-way approach of the paper addresses this issue efficiently. Firstly, it discards unrolling factors of lags that are dominated in terms of the delay. Secondly, it gives priority to the lags, which brings in the maximum gains. Thirdly, it assigns the weights which depend on the reduction in lags relative to the additional delays in the finite state machine: hence the limited amount of lags gets distributed among the different loops. The final result is that lots of candidates get checked, although the complete synthesis only happens for the best few of them [2]. The authors state that the average margin of error in prediction of lags is 7% and estimation takes less than five minutes according to their experiments. This shows that the idea is really about selecting the unrolling factors in a timing-aware way and not just doing unrolling.

IV. Vitis HLS Application Example

A. Experimental Method

A 32-element integer dot product was used:

```
#define N 32
extern "C" int dot_product(int x[N], int z[N]) {
   int q = 0;
loop_main:
   for (int i = 0; i < N; ++i) {
#pragma HLS UNROLL factor=U
      q += x[i] * z[i];
   }
   return q;
}
```

The C simulation generated the expected results in advance of synthesis. The goal was an Artix-7 device with a 10 ns target frequency and 2.70 ns clock uncertainty. The factors 1,2,4,8,16,32 were synthesized using normal array storage. The second experiment was done with complete separation of arrays:

```
#pragma HLS ARRAY_PARTITION variable=x complete dim=1
#pragma HLS ARRAY_PARTITION variable=z complete dim=1
```

The complete partitioning leads to the use of independently accessible storage elements, and thus, the problem of ports limitation has been removed [1], [4]. The data obtained were expressed in terms of top latency, II, trip count, clock estimation, timing slack, number of DSPs used, number of FFs used, and number of LUTs used in total.

B. Measured Results

The ordinary memory experiment shows the results provided in Table I. Factor 2 provides considerable development, since the latency drops from 370 ns to 210 ns, whereas II still equals 1. Factors 4, 8, and 16 result in the continuing decrease of the trip count while increasing of II to respectively 2, 4, and 8. This way, the latency does not exceed the value of

TABLE I
ORDINARY-MEMORY VITIS HLS RESULTS

U	Lat. ns	II	Trip	DSP	FF	LUT
1	370	1	32	3	311	166
2	210	1	16	6	573	229
4	220	2	8	6	832	508
8	220	4	4	6	835	603
16	220	8	2	6	979	816
32	220	–	–	6	1025	978

TABLE II
COMPLETE-PARTITIONING VITIS HLS RESULTS

U	Lat. ns	II	Trip	DSP	FF	LUT
4	130	1	8	12	1159	671
8	90	1	4	24	2203	1007
16	80	1	2	48	4486	2457
32	50	–	–	60	4234	2150

220 ns. Full unrolling only requires 6 DSPs; however, the architecture is still unable to achieve all products through concurrent accesses, thus the estimated clock dropped to 7.826 ns with calculated slack of about -0.53ns after uncertainty.

In case of partitioned arrays, II again remains equal to 1 for factors 4-16, while the latency reduces to 130-90-80 ns. As for full unrolling, it makes it possible to achieve 50 ns. However, this progress grows hand in hand with fast resource expansion, since factor 32 uses 60 DSPs, 4234 FFs and 2150 LUTs, which can be seen in Table II.

C. RTL Interpretation and Relation to the Main Paper

Figure 3 presents a structural analysis through simplified Vivado RTL illustrations. The rolled design corresponds to one multiplier being utilized under FSM and multiplexer control. Additionally, factor-4 and factor-8 illustrations make copies of multipliers and adders forming a larger reduction tree. The unrolled illustrations seem to be tidy but require more arithmetic hardware. In contrast, the rolled illustration needs additional control and selection logic since the smaller datapath is reused repeatedly.

D. Timing-Slack and Report Interpretation

The HLS timing report compares the target clock with the estimated combinational delay and a clock-uncertainty allowance. For this experiment, an approximate margin is

$$S = T_{\text{target}} - T_{\text{estimated}} - T_{\text{uncertainty}}. \tag{7}$$

Positive slack means that time is left, while negative slack shows that the desired target cannot be reached. The ordinary-memory factors of 1 and 2 yield a slack time of approximately +0.44 ns, factors between 4 and 16 roughly +0.09 ns, while the slack time factor of 32 corresponds to -0.53 ns. The partitioned factors exceed the estimates, but the positive slack time for partitioned factors from 8 to 32 is merely +0.09 ns.

It is also significant to differentiate between the report interval of the highest level and the loop II. The highest level function interval involves invocation and control behavior of

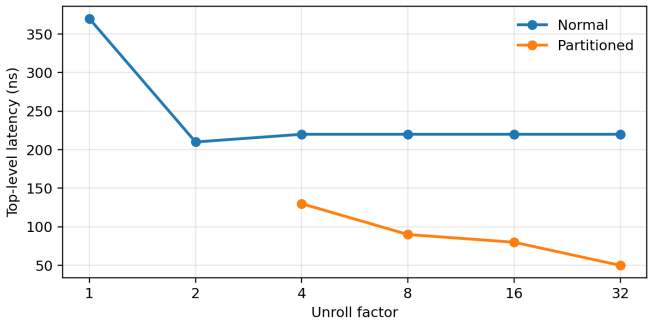

Fig. 1. Top-level latency. Ordinary memory reaches a plateau after factor 2; complete partitioning permits further reductions.

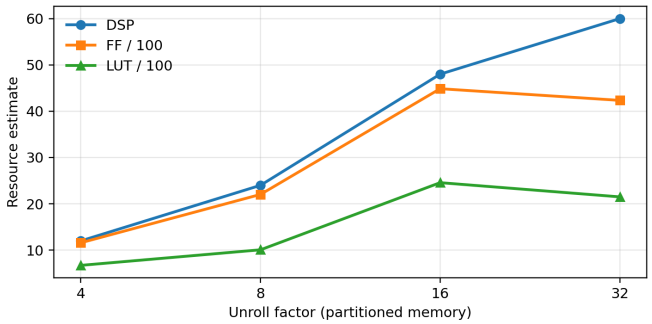

Fig. 2. Resource growth for complete array partitioning. FF and LUT values are divided by 100 for display.

the overall component while the loop line tells us how many times a pipelined loop will start a new iteration. Figure 4 shows valuable examples. The report on ordinary memory factor-4 states the limitation of resource availability and gives Loop II $= 2$; while the report on the partitioned memory factor-8 states the Loop II $= 1$ while using 24 DSP. This is confirmation of a statement that ordinary memory could not feed the new expanded loop.

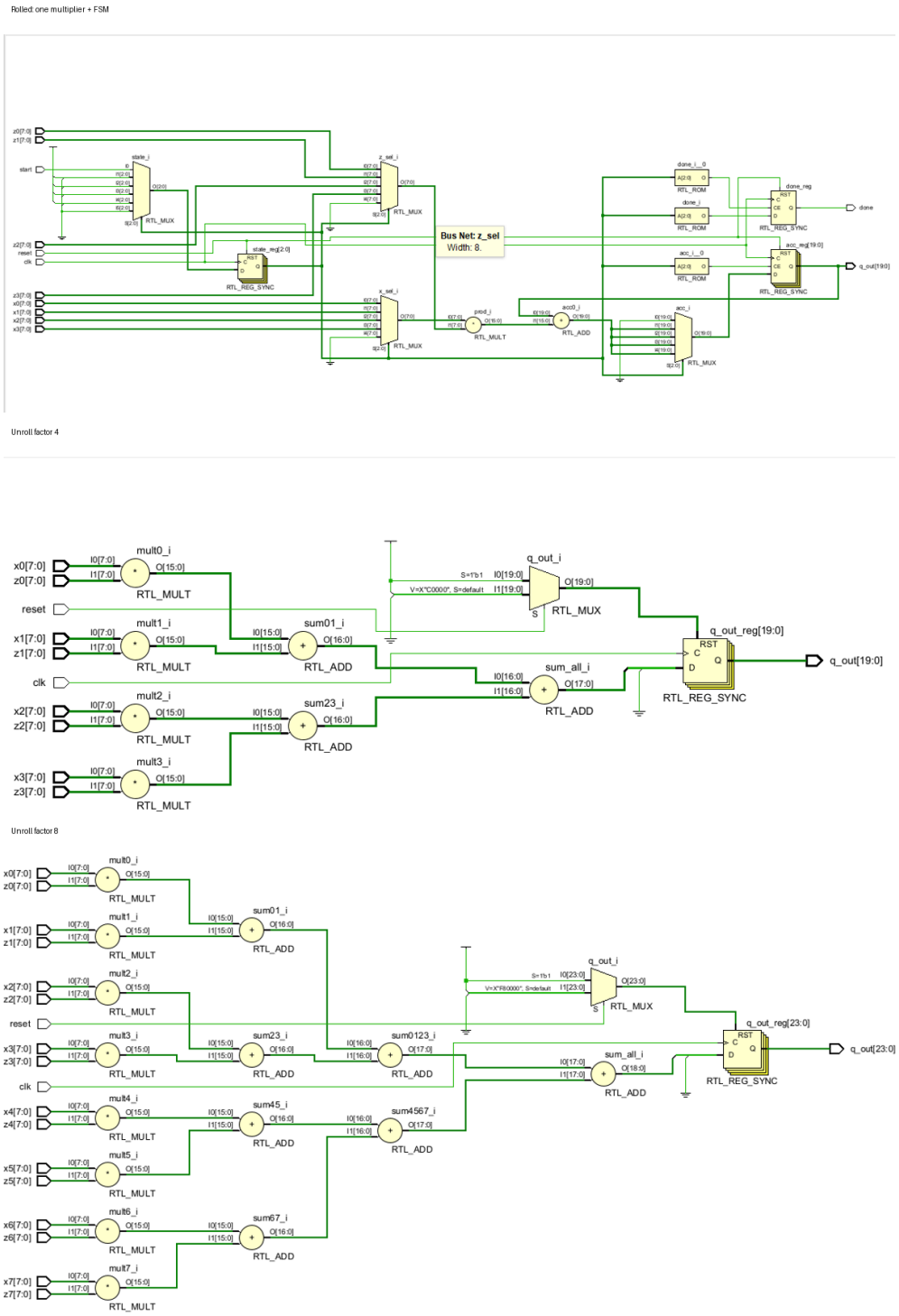

Fig. 3. Simplified RTL visualization: rolled implementation, unroll factor 4, and unroll factor 8. These hand-written RTL examples illustrate structural growth; the quantitative measurements come from Vitis HLS.

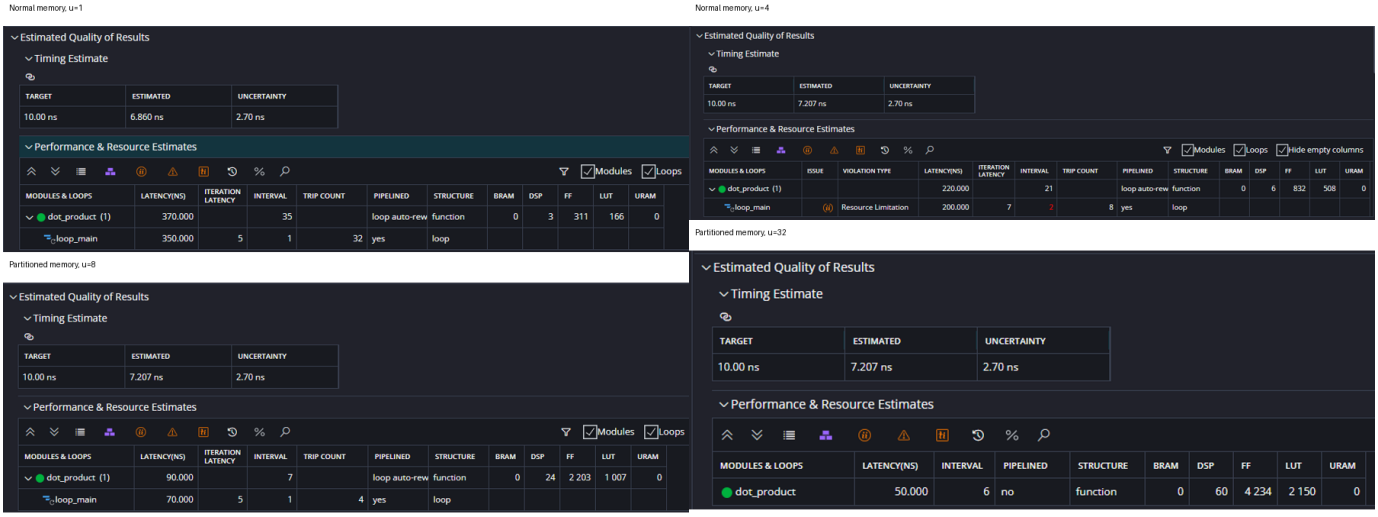

Fig. 4. Representative Vitis HLS reports used to record timing, II, trip count, and resource estimates.

The experiment backs the large conclusion of the paper but does not reproduce the controller delay value directly. Vitis provides total scheduling, latency, clock and resource evaluation; but does not provide the precise calibrated D_{FSM} value used for finding the mentioned Equation 5. Thus, the increase of LUT/FF number, decrease of slack, II violations and increase of RTL value support the assumptions about the growing pressure of implementation but do not measure the controller delay value. The ordinary memory results show that the architecture of memory influences performance very much. Unrolling opens new parallel works, yet the works are only useful if the datapath is capable of providing them.

V. Discussion

The findings show various concepts regarding cost. First of all, it was found that the loop control overhead decreases proportionally to N/u. Secondly, the hardware necessity increases if processes are repeated rather than shared among various units. Lastly, the complexity of HLS design increases due to various operations also being done, thus bringing no extra advantages.

In the experiments carried out, the second factor was found to be optimal for the standard memory use as it achieves almost twice lower latency with no significant resource increase compared to its higher partitioned factors. The firth factor gives better results due to the lower cost. The factor 32 is the best only when lag time saves all the area occupied by 60 DSP and units with higher performance. These factors depend on a particular design and are not optimal for other. The methods utilized in this article allow turning the process of choosing a proper factor into a mathematical formulas.

Another limitation is that, although the dot product has standard fixed bounds, this possesses varying dependencies through q. There may be variations in the output due to different kernels and technologies and memory interfaces as well as different clock targets. Post-place-and-route timing should also be more accurate than the figure provided at HLS. Comparable optimizations of loops and memory can be found in Intel oneAPI, LegUp, and Bambu framework but differ in how they are reported and scheduled According to the research [3]–[5].

VI. Conclusion

The idea behind loop unrolling in HLS is not to guarantee speed but rather to reduce the amount of loops and enable parallelism. In addition, it increases scheduling, which also means increasing states of FSMs, bits of states, number of multiplexers, resources, and delays. The team around Kurra found a way out of that problem by calculating just how many states have to be spent on FSM for any certain situation and applying calculations in synthesis process without going through the whole synthesis procedure with all calculations.

The example of the state Vitis HLS application demonstrates a similar engineering choice as well. Simple memory gave results only at the two times speed increase. Bigger numbers reduced total number of trips but in the same time increased the interval of time between each trip thus leaving the latency on the same level. The full array partitioning made the latency equal to 50 ns but the cost of that was 60 DSPs and plenty of FFs/LUTs. So, the best unroll number is the one that is able to increase speed but the conditions related to controller timing and resources of system must be kept.

References

[1] AMD, *Vitis High-Level Synthesis User Guide, UG1399*, v2025.2 ed., Advanced Micro Devices, Inc., Jan. 2026, implementation reference. It documents the Vitis HLS flow, loop-unroll and array-partition pragmas, scheduling/binding, C synthesis, and timing/resource reports used in the experiment.

[2] S. Kurra, N. K. Singh, and P. R. Panda, "The impact of loop unrolling on controller delay in high level synthesis," in *Design, Automation and Test in Europe Conference and Exhibition (DATE)*. EDAA, 2007, main reference. It studies how loop-unroll factors change HLS controller delay, proposes a high-level delay estimate, and uses the estimate to select factors under an FSM-delay budget.

[3] Intel, *Intel oneAPI FPGA Handbook*, Intel Corporation, 2024, related modern-tool reference. It describes how source code maps to FPGA datapaths, loop unrolling, loop analysis, schedule viewers, memory optimization, area estimates, and timing failures.

[4] Microchip Technology Inc., *LegUp 2021.1 Documentation: User Guide*, 2021, related academic HLS reference. It covers loop-level parallelism, loop pipelining, memory partitioning, pragmas, and software/hardware co-simulation.

[5] Politecnico di Milano, "Bambu hls framework," PandA Project website, 2021, related open-source HLS framework. It provides context for alternative tools that synthesize high-level programs into hardware.